

Bitácora técnica: etiquetador FST y CNN

26 de septiembre de 2026 · Angel

Se construyó un etiquetador automático que convirtió 14 videos heterogéneos del test de nado forzado en 3100 bloques etiquetados (más 96 de conducta activa), y con ellos se entrenó una CNN 3D que alcanza kappa +0.625 validando por video. Esta bitácora documenta el objetivo, el método, cada decisión medida y los resultados.

1. Objetivo

El objetivo final de la tesis es una **CNN que clasifique la conducta de ratas en el test de nado forzado (FST)** a partir de video. Para entrenarla hace falta un conjunto de datos etiquetado, y ese conjunto no existía.

El problema tenía tres restricciones:

1. **No hay anotaciones de expertos.** El estudiante es el etiquetador de referencia.
2. **Los videos ya estaban grabados, y de forma heterogénea:** distintos días, encuadres, fondos, iluminación, resoluciones y hasta orientación de cámara. Etiquetar a mano 5 minutos por rata y por video, en bloques de 5 s, es inviable a escala.
3. **Riesgo de sobreajuste al fondo.** Si el clasificador aprende el aspecto de cada video (pizarrón, luz, encuadre) en lugar de la conducta, falla en videos nuevos. Toda la validación se diseñó para detectarlo.

Por eso el trabajo se dividió en dos etapas:

- **EtapA A, el etiquetador (fst_auto):** una herramienta de visión clásica que mide al animal y propone etiquetas, más una interfaz de revisión humana. Su producto es el dataset.
- **EtapA B, la CNN (cnn/):** el clasificador final de la tesis, entrenado con ese dataset y comparado contra el etiquetador.

El repositorio es `ErickJester/fst-etiquetador` (privado), rama `main`.

2. Marco experimental y definiciones operativas

Cada video muestra hasta 4 ratas, cada una en su cilindro con agua. La unidad de análisis es el **bloque de 5 s por espécimen** (`BLOCK_S = 5.0`), siguiendo el muestreo por intervalos validado por Álvarez-Suárez et al. (2015). Un video de 5 min con 4 ratas produce unos 240 bloques.

Se usa el esquema de tres conductas de Detke, Rickels y Lucki (1995), con las definiciones operativas de Craft, Kostick, Rogers y Tsutsui (2010):

Conducta	Definición operativa	Señal que la distingue
Inmovilidad	Flota pasiva; solo los movimientos necesarios para mantener el hocico fuera del agua. No cambia de cuadrante.	Poco desplazamiento y poca energía de movimiento
Nado	Se desplaza cruzando al menos 2 de los 4 cuadrantes del cilindro. Bucear cuenta como nado.	Desplazamiento horizontal, varios cuadrantes
Escalamiento	Movimientos rápidos de las patas delanteras contra la pared, rompiendo la superficie.	Postura vertical, cabeza alta, pataleo rápido junto a la pared

Conducta activa. Se añadió una cuarta etiqueta, `activa` = *no inmóvil, sin decidir entre nado y escalamiento*. Corresponde al nivel original de Porsolt (*inmóvil* / *no inmóvil*); Detke separó después lo activo en *nado* y *escalamiento*. Es una etiqueta más gruesa pero fiable, válida para la medida clásica de Porsolt. Queda fuera del modelo de 3 clases y entra en una segunda métrica: *inmóvil* contra *activa*.

Regla de propuesta (idéntica en el etiquetador y en la CNN), con umbral $U = 0.80$:

- 1. Si una conducta tiene probabilidad $\geq U \rightarrow$ esa conducta.
- 2. Si no, pero $p(\text{nado}) + p(\text{escalamiento}) \geq U \rightarrow$ `activa`.
- 3. Si ninguna $\rightarrow$ bloque dudoso (`usar = 0`), para revisión humana.

En `IMG_0840` esta regla bajó los dudosos de 23 a 7.

3. Datos

El conjunto final tiene **14 videos etiquetados, 3197 bloques** (3101 con una de las 3 conductas y 96 `activa`). Un 15.º video, `AcuN1`, se reservó como prueba externa (sección 11).

Video	Bloques etiquetados	Nado	Inmovilidad	Escalamiento	Activa
AcuBN1	296	204	64	24	4
AcuBN2	292	162	103	26	1
IMG_0839	292	190	36	60	6
AcuYN1	288	151	112	19	6
AcuYN2	288	228	39	14	7
Acu-YBN-1	256	185	56	11	4
IMG_0826	240	106	97	20	17
acu-yintang1	195	74	90	22	9
Acu-YBN-2	192	93	80	16	3
Acu-baihui1	189	99	15	73	2
acu-yintang2	189	133	23	27	6
Acu-baihui2	186	125	14	33	14
IMG_0840	148	77	17	38	16
IMG_0837	146	102	13	30	1
Total	3197	1929	759	413	96

El entrenamiento del etiquetador usa 3196: descarta un bloque de escalamiento en que el animal no fue visible y sus rasgos quedaron vacíos.

El reparto está desbalanceado: nado es el 62% de los bloques de 3 clases y escalamiento solo el 13%. Los videos incluyen animales con y sin tratamiento; eso cambia **cuánto** aparece cada conducta, no cómo se ve, y es deseable porque aporta ejemplos de las tres.

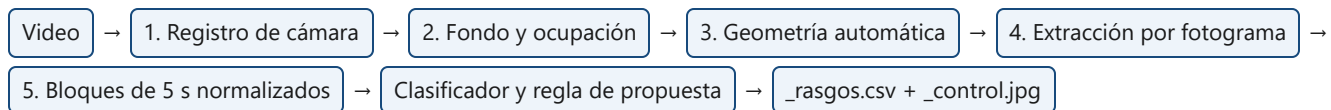
Heterogeneidad y problemas de grabación encontrados:

- **Resolución distinta:** los `IMG_08xx` son 1920×1080; `Acu-baihui1` es 1280×720.

- **Cámara que se mueve:** en `IMG_0840` el encuadre se desplaza hasta 574 px en vertical a lo largo del video.
- **Cámara girada:** `IMG_0846` se grabó con la cámara de lado y aparece en vertical (1080×1920).
- **Toma demasiado cercana:** en `IMG_0846` los bordes superior e inferior del aparato quedan fuera del cuadro.
- **Archivos dañados:** una copia de `Acu-baihui1` estaba trunca ("partial file"); `IMG_0829` no produjo rasgos.
- **Experimentador en cuadro:** en el primer bloque de cada rata suele verse a la persona metiéndola al agua (menos del 2% de los clips).
- **Distractores en el fondo:** pizarrón con rotulador rojo del mismo gris que el aparato, reflejos y gotas en el acrílico.

4. Arquitectura del etiquetador

El etiquetador es visión por computadora clásica (OpenCV + NumPy), sin redes neuronales y sin GPU. `etiquetar.py` ejecuta cinco etapas definidas en `lib.py`; un video de unos 6 min tarda alrededor de 16 min en CPU con `--paso 12`.



1. Registro de cámara (registrar). Cada fotograma se alinea contra uno de referencia tomado a la mitad del video. Se detectan 2000 puntos ORB (`fastThreshold=7`) sobre la imagen en gris reducida a la mitad, se emparejan con fuerza bruta en distancia de Hamming y prueba de razón de Lowe (0.75), y se estima una transformación de **similitud** (traslación, rotación y escala) con RANSAC (umbral 2.5 px, 3000 iteraciones, mínimo 12 pares). La cámara de este montaje cambia hasta ~5% de escala, así que la traslación sola dejaba el fondo borroso. Si la referencia no se puede leer, se reintenta al 35, 20, 10 y 2% del video (archivos truncados).

2. Fondo y mapa de ocupación (fondo_y_ocupacion). Con un fotograma alineado por segundo se calcula el fondo como el **percentil 10 temporal** de cada píxel. Como la rata es clara sobre agua oscura, el percentil bajo la borra y deja el aparato vacío; la mediana no sirve, porque un animal inmóvil queda incrustado en el fondo. Después se acumula la fracción de tiempo en que cada píxel es "rata": diferencia contra el fondo mayor a 45 niveles (`DIFF_THR`) y brillo absoluto mayor a 80 (`BRIGHT_THR`).

3. Geometría automática (detectar_geometria). Deduce de cada video las columnas de cada tubo, la línea de agua y el fondo del cilindro, sin medidas escritas a mano. El orden es deliberado:

1. **Filas primero:** de los bordes horizontales del fondo en todo el cuadro, los dos más fuertes entre el 20 y el 95% de la altura son los rebordes del aparato; entre ellos, el pico interior superior es el agua y el inferior, el suelo.
2. **Columnas después:** la ocupación solo dentro de la franja de agua da rachas de actividad; se descartan las que tocan el borde de la imagen y, con `--tubos N`, se conservan las N con más actividad.
3. **Tubos de los extremos** se ensanchan al ancho típico: la rata trepa pegada a la pared exterior.
4. **Refinado de filas** usando solo la banda estrecha de las rachas detectadas.

El resultado se dibuja en `<video>_control.jpg` (azul = agua, verde = fondo, amarillo = zona medida) para verificación visual obligatoria.

4. Extracción por fotograma (extraer). Cada 12 fotogramas (`--paso 12`) y en cada tubo se segmenta la rata (umbral contra el fondo, apertura 5×5, cierre 9×9, mayor componente conexa). Para medir la postura se erosiona con un elemento elíptico de 21 px, que borra la cola, y se calculan los momentos de imagen del núcleo. Las

medidas dinámicas (energía de movimiento, IoU, Dice) se comparan contra el fotograma más cercano a **0.40 s antes** (DT_REF), no contra el anterior, para que no dependan del paso.

5. Agregación en bloques (bloques). Las medidas se agrupan en bloques de 5 s y se normalizan (sección 5). Se descarta el último bloque si tiene menos del 60% de los fotogramas de un bloque típico.

Mapeo de vuelta al video crudo (cajas_crudas). Todo el análisis vive en el marco estabilizado. Para dibujar sobre el video original se deshace el registro con la inversa de la matriz afín del fotograma a la mitad de cada bloque, y, si hubo --rotar, se deshace también el giro (sin_girar). El resultado son las columnas bx0, by0, bx1, by1, que usan el revisor y la CNN.

5. Rasgos y normalización

Cada bloque se describe con 15 rasgos (lib.RASGOS). Las distancias se dividen por el **largo del cuerpo**, las alturas por la **altura de la columna de agua** y las velocidades por **segundo**. Es lo único que hace comparables videos grabados de cerca y de lejos: sin ello el clasificador aprende el encuadre, no la conducta.

El largo del cuerpo (escala) es 4σ del eje mayor de la silueta sin cola, tomado como mediana de **todo el video**, no de cada bloque, para que el ruido de un bloque no altere su propia normalización. El centroide se suaviza con una mediana móvil de 5 muestras antes de medir trayectorias.

Rasgo	Qué mide	Unidad	Conducta que ayuda a separar
nq	Cuadrantes del cilindro visitados con permanencia ≥ 0.20 s (DWELL_S)	0 a 4	Nado (≥ 2) contra inmovilidad
rng	Diagonal del rectángulo que abarca el centroide	largos de cuerpo	Nado
spanx	Recorrido horizontal del centroide	largos de cuerpo	Nado
spany	Recorrido vertical del centroide	largos de cuerpo	Escalamiento, buceo
path	Longitud de la trayectoria por segundo	largos de cuerpo / s	Nado
me	Energía de movimiento: diferencia media de intensidad dentro del animal contra 0.40 s antes	niveles de gris	Actividad general
me_max	Máximo de me en el bloque	niveles de gris	Pataleo breve
vert	$ \sin \theta $ del eje mayor (1 = vertical)	0 a 1	Escalamiento
elong	Razón entre ejes principales de la silueta	adimensional	Postura estirada o encogida
hrise	Altura máxima de la cabeza sobre el agua	columnas de agua	Escalamiento
hmean	Altura media de la cabeza sobre el agua	columnas de agua	Escalamiento
above	Área de la rata por encima de la línea de agua	largo ²	Escalamiento
area	Área visible de la rata	largo ²	Buceo, oclusión
iou	Intersección sobre unión de la silueta contra 0.40 s antes	0 a 1	Inmovilidad (alto)
dice	Coefficiente de Dice de la silueta contra 0.40 s antes	0 a 1	Inmovilidad (alto)

Decisiones de medición que no deben deshacerse:

- La energía de movimiento se mide **solo dentro del animal** (máscara dilatada 15×15), no en todo el tubo: el oleaje del agua satura la medida.
- La cola se **erosiona antes de medir la postura**; sin eso los momentos miden la cola y no la verticalidad del cuerpo.
- `DT_REF` y `DWELL_S` están fijados en **tiempo**, no en fotogramas, para que cambiar `--paso` o los fps no cambie los valores.
- Si el animal se ve en menos de 5 muestras del bloque, sus rasgos quedan vacíos y el bloque no se usa para entrenar.

6. Clasificador del etiquetador y validación

El clasificador es un `HistGradientBoostingClassifier` de scikit-learn (boosting de árboles por histogramas) sobre los 15 rasgos: `max_iter=300`, `learning_rate=0.06`, `max_leaf_nodes=15`, `l2_regularization=1.0`, `random_state=0`. Se eligió porque tolera valores faltantes, no necesita escalar los rasgos y entrena en segundos en CPU. En conversaciones previas se le llamó "Random Forest" de forma informal; el modelo real es este.

Entrenamiento (`entrenar.py`). Cruza cada `x_rasgos.csv` con su `x_mano.csv` por (bloque, especimen):

- Antes de cruzar, borra `clase`, `confianza` y `usar` del CSV de rasgos. Si no, pandas renombra a `clase_x/clase_y` y la etiqueta humana se pierde.
- Descarta las filas de confianza `baja` (salvo `--usar-dudosas`) y las de clase vacía ("no sé" del revisor). Con pandas 3.0 hace falta `fillna('')` antes de `astype(str)`.
- Rechaza clases mal escritas y **mezclas de** `--paso` distintos.
- Las filas `activa` quedan fuera del modelo de 3 clases.

Validación. `GroupKFold` con 5 partes, agrupando **por video**: cada video se predice con un modelo que nunca lo vio. Con un solo video se agrupa por animal, y el script avisa que la nota sale optimista. Agrupar por bloque sería inválido: dos bloques seguidos del mismo animal son casi el mismo fotograma.

La salida reporta la línea base (adivinar la clase más común), exactitud, kappa de Cohen, acierto por clase y matriz de confusión; la **métrica Porsolt** (inmóvil contra activa, incluyendo las filas `activa`, con su propio `GroupKFold` manual); y una tabla de exactitud frente a cobertura al descartar bloques por debajo de un umbral de confianza.

El modelo guardado (`modelo_fst.joblib`) es un diccionario con el clasificador, la lista y el orden de sus columnas, las clases, el `paso`, el número de bloques y los videos usados. `etiquetar.py` compara ese `paso` con el pedido **antes** de procesar ningún video y se niega si no coinciden.

7. Revisión humana y flujo en bola de nieve

La revisión se hace en `revisor.html`, una página autónoma (881 líneas, HTML + JavaScript, sin servidor) que se abre con doble clic en Chrome o Edge. Sustituyó a `revisar.py`, que generaba videos cortos por bloque.

Entrada. El video crudo y el `x_rasgos.csv` (o `x_cnn.csv`), arrastrados o elegidos con selectores. También acepta `?video=&csv=` en la URL, con `fetch` sin caché.

Interfaz.

- Video casi a pantalla completa. El tubo del bloque actual se resalta con un marco dorado y cuatro velos oscuros alrededor, posicionados con `bx0..by1` (se usan cuatro `div` porque Chrome deja de pintar un `box-shadow` gigante sobre un `<video>` en reproducción).
- Línea de tiempo por espécimen, coloreada por la propuesta.
- Botones y teclas: **1** nado, **2** inmovilidad, **3** escalamiento, **4** conducta activa, **R** repetir el bloque, **0** no seguro, **Espacio** pausa, **Ctrl+S** guardar.
- Flechas: una pulsación va al bloque contiguo; dos en menos de 400 ms saltan al siguiente dudoso.

Modos.

Modo	Qué muestra	Para qué
Dudosos	Solo se detiene en los bloques con <code>usar = 0</code>	Revisión rápida en bola de nieve
Todos	Recorre todos los bloques con la propuesta visible	Auditoría completa
Desde cero	Oculto todo rastro de la máquina (propuesta, colores, contadores)	Etiquetado independiente, para medir

El progreso se guarda en `localStorage` por video, número de filas y modo; el modo desde cero guarda aparte.

Salida. `X_mano.csv` con `bloque`, `especimen`, `inicio_s`, `propuesta`, `seguridad`, `clase`, `confianza`, `origen`. La columna `origen` registra cómo nació cada etiqueta: `humano_confirma`, `humano_corrige`, `humano_duda` (casilla vacía), `humano_ciego` (modo desde cero) o `maquina` (bloque seguro que no se revisó).

Flujo en bola de nieve. No se etiqueta cada video entero a mano:

1. Se puntúa a mano un video semilla (`IMG_0826` , 240 bloques).
2. Se entrena con él.
3. El modelo propone etiquetas para el siguiente video; se revisan en el modo "dudosos".
4. Se guarda, se reentrena con todos los videos acumulados y se repite.

Cada reentrenamiento parte de cero con **todos** los CSV, viejos y nuevos: el modelo no se actualiza incrementalmente. El primer video costó cerca de una hora; con el modelo maduro, un video se revisa en minutos porque solo el 2-3% de los bloques queda como dudoso.

8. Resultados del etiquetador

Con 14 videos, validando por video, el etiquetador alcanzó **exactitud 0.982 y kappa +0.966** en las 3 conductas, y 0.987 (kappa +0.963) en la métrica Porsolt. Cada ronda de videos nuevos se validó igual; el rendimiento subió y luego se estabilizó.

Ronda	Videos	Bloques (3 clases)	Validación por	Exactitud	Kappa	Ejemplos de escalamiento	Porsolt (kappa)	Exactitud con umbral 0.80 (cobertura)
Semilla	1 (<code>IMG_0826</code>)	100 (confianza alta y media)	animal	0.880	—	14	—	0.929 (85%)
2	3	—	video	0.899	+0.826	—	0.948	0.953
3	7	1458	video	0.976	+0.955	172	0.978 (+0.944)	0.990 (97%)
4	10	2086	video	0.977	+0.957	272	0.983 (+0.955)	0.988 (98%)

Ronda	Videos	Bloques (3 clases)	Validación por	Exactitud	Kappa	Ejemplos de escalamiento	Porsolt (kappa)	Exactitud con umbral 0.80 (cobertura)
5	14	3100	video	0.982	+0.966	412	0.987 (+0.963)	0.990 (98%)

La semilla se midió con 0.890 (kappa +0.819) antes de fijar `--paso 12` y con 0.880 después; la tabla usa el valor con el paso definitivo. La línea base (adivinar siempre nado) es 0.622 con 14 videos.

Acierto por clase con 14 videos: escalamiento 0.977, inmovilidad 0.970, nado 0.988.

Etiqueta \ modelo	Escalamiento	Inmovilidad	Nado
Escalamiento	397	7	8
Inmovilidad	1	738	20
Nado	3	17	1909

La confusión residual está casi toda entre nado e inmovilidad (37 bloques de 3100): es la frontera borrosa entre flotar y nadar despacio. El escalamiento, la conducta más débil con un solo video (f1 0.786, 14 ejemplos), dejó de serlo al llegar a 412 ejemplos: se arregló con datos, no con código.

Cómo leer estas cifras. En las rondas 3 a 5 la mayoría de los bloques conservó la propuesta del propio etiquetador, y solo se revisaron a mano los dudosos. Por eso estos números miden sobre todo la **consistencia** del etiquetador entre videos que no vio al entrenar, y no son una meta directamente comparable para otro método. Una medida independiente requiere un video etiquetado en modo "desde cero" (sección 11).

9. Decisiones medidas y problemas resueltos

9.1 El paso de muestreo: más detalle no es mejor

`--paso N` analiza 1 de cada N fotogramas. Sobre `IMG_0826`, con los mismos bloques y etiquetas:

Métrica	<code>--paso 12</code>	<code>--paso 2</code>
Exactitud	0.880	0.770
f1 escalamiento	0.786	0.476
Exactitud con umbral 0.80	0.929	0.857

Causa: `path` y `rng` acumulan el temblor del centroide. Con 75 muestras por bloque en lugar de 12, ese temblor se suma seis veces: `path` sale 2.06 veces mayor con paso 2 sin que la rata se haya movido el doble.

Solución: las medidas dinámicas se anclaron en tiempo (`DT_REF = 0.40 s`, `DWELL_S = 0.20 s`). Después del arreglo, la razón `paso 2 / paso 12` es 1.00 en `me`, `iou`, `dice`, `vert`, `area` y `elong`; siguen dependiendo `path` (2.06), `rng` (1.29), `spanx` (1.23) y `nq` (1.11). Se fijó `--paso 12` por defecto, el modelo guarda su paso y `etiquetar.py` rechaza otro antes de procesar.

9.2 Registro de problemas

Problema	Causa	Solución	Efecto
En <code>IMG_0840</code> un "tubo" abarcaba todo el cuadro	Bordes del entorno dominaban la detección de columnas	Filas primero; columnas solo dentro de la franja de agua; descartar rachas que tocan el borde	Tubos correctos en todos los videos probados
El suelo del tubo se confundía con el borde de la mesa	Buscar de pared a pared metía bordes ajenos	Refinar filas solo en la banda estrecha de las rachas	Error del suelo de 45 px a 16 px
Tubo del extremo recortado	La rata trepa pegada a la pared exterior	Ensanchar los tubos de los extremos al ancho típico	f1 escalamiento de 0.55 a 0.79
Rotulador rojo del pizarrón	Mismo gris que el aparato	Geometría desde el mapa de ocupación, no desde un umbral	—
Registro por correlación de fase y perfiles de intensidad	Solo modelan traslación; la escala cambia ~5%	ORB + RANSAC con similitud	Fondo nítido
Recuadro fuera de la rata en el revisor	Cámara que se mueve hasta 574 px (<code>IMG_0840</code>)	<code>cajas_crudas</code> : inversa del registro a mitad de cada bloque	Recuadro sobre la rata todo el video
El navegador mostraba un CSV viejo	Caché de <code>fetch</code>	<code>cache: 'no-store'</code>	—
Videos grabados con la cámara girada	El fotograma llega de lado	<code>--rotar</code> (derecha/izquierda/180), envoltorio <code>_Video</code> , <code>sin_girar</code> para volver al video crudo	Mapeo exacto verificado a 90, 180 y 270 grados
<code>IMG_0846</code> sigue fallando tras girarlo	Toma tan cercana que los rebordes quedan fuera del cuadro	Pendiente	Video excluido
El modelo no cargaba en otra PC	Guardado con numpy 2; numpy 1.x falla con un error de PCG64	<code>numpy>=2.0</code> y scikit-learn 1.9 en <code>requirements.txt</code> ; <code>comprobar.py</code> lo detecta	—
La etiqueta "no sé" rompía el entrenamiento	En pandas 3.0 <code>astype(str)</code> conserva los nulos	<code>fillna('')</code> antes	—
La etiqueta humana desaparecía al cruzar	Columnas <code>clase</code> duplicadas (<code>clase_x</code> / <code>clase_y</code>)	Borrar las columnas de la máquina antes del cruce	—
Velo del revisor invisible al reproducir	Chrome no pinta un <code>box-shadow</code> enorme sobre un <code><video></code> activo	Cuatro <code>div</code> de velo	—
La semilla de <code>IMG_0826</code> se reemplazó	Una exportación de <code>revisor.html</code> se guardó encima	El original, con justificación por bloque, queda en el commit <code>f63c67a</code> ; 57 de 240 bloques difieren	Sin efecto en el entrenamiento

10. CNN: datos, arquitecturas y resultados

La mejor CNN, una **R(2+1)D-18 preentrenada en Kinetics-400** (versión 4), obtuvo exactitud 0.804 y kappa +0.625 validando por video, y 0.896 (kappa +0.703) en la métrica Porsolt. Se llegó a ella en cuatro versiones, cada una medida con el mismo protocolo.

10.1 Preparación de los clips (`cnn/recortar.py`)

Corre en la PC donde están los videos, en CPU, alrededor de 1 min por video. Por cada bloque de `X_mano.csv`:

1. Toma el recuadro del tubo en coordenadas del video crudo (`bx0..by1` del `x_rasgos.csv`), que ya compensa el movimiento de cámara.
2. Extrae N fotogramas repartidos uniformemente en los 5 s, en los instantes $t_0 + (k + 0.5) \cdot (t_1 - t_0) / N$. El video se lee **una sola vez, de corrido** (`grab` para los fotogramas que no se usan), mucho más rápido que saltar con `set` en archivos .MOV.
3. Recorta el tubo, lo endereza si el video llevaba `--rotar` y lo reduce a **gris de 128×96 px** (alto × ancho, `INTER_AREA`).
4. Descarta los bloques con algún fotograma ilegible.
5. Guarda `clips/X.npz` (arreglo `uint8` de forma $N_{\text{bloques}} \times N \times 128 \times 96$, más clase, confianza, espécimen, bloque y video) y una tira `x_muestra.jpg` para verificar a ojo que la rata cae dentro del recorte.

Se generaron primero con $N = 8$ y después con $N = 16$. El conjunto tiene **3179 clips de 14 videos** (3083 de 3 clases). La carpeta se sube a Google Drive (`Mi unidad/fst/clips`) y no se versiona en git.

10.2 Entrenamiento (`cnn/fst_cnn.py` y `cnn/fst_cnn.ipynb`)

El código de la red vive en `fst_cnn.py`; el cuaderno de Colab solo lo llama. Como el repositorio es privado, el cuaderno no lo clona: lee `fst_cnn.py` y los clips desde Drive. Cada versión imprime `fst_cnn version N cargada` para confirmar que Colab usa la copia nueva. Se entrenó en **Colab Pro con GPU NVIDIA T4** y precisión mixta (AMP).

Comunes a todas las versiones:

- **Validación:** `GroupKFold` de 5 partes por video, con la misma salida que `entrenar.py`, para comparar directo.
- **Aumento de datos:** brillo $\times U(0.7, 1.3) + U(-0.1, 0.1)$, volteo horizontal con $p = 0.5$ y desplazamiento aleatorio de hasta ± 6 px.
- **Pérdida:** entropía cruzada con suavizado de etiquetas 0.05 y peso por clase. **Optimizador:** AdamW (decaimiento $1e-4$) con OneCycleLR.
- Las filas `activa` no entrenan, pero sí se predicen para la métrica Porsolt: inmóvil si $p(\text{inmovilidad}) \geq 0.5$.

Versión	Red	Entrada	Entrenamiento
v1	ResNet18 (ImageNet)	3 fotogramas crudos como canales RGB; al entrenar, un trío al azar con separación aleatoria; al predecir, promedio de tríos fijos	12 épocas, lote 64, lr $3e-4$, peso de clase 1/frecuencia
v2	ResNet18 (ImageNet)	Igual que v1	40 épocas; peso de clase $\sqrt{1/\text{frecuencia}}$ normalizado
v3	ResNet18 (ImageNet)	Mapas de movimiento: (1) fotograma central estandarizado por su media y desviación, la postura; (2) desviación estándar temporal de cada píxel, el desplazamiento; (3) diferencia absoluta media entre fotogramas seguidos, el pataleo. Canales 2 y 3 escalados por 0.08 y recortados a $[-1, 6]$	40 épocas, lote 64, lr $3e-4$
v4	R(2+1)D-18 (Kinetics-400)	Los 16 fotogramas como video ($3 \times 16 \times 128 \times 96$), gris replicado en 3 canales, cada clip estandarizado por su propio brillo	15 épocas, lote 16, lr $1e-4$

10.3 Resultados

Versión	Fotogramas	Exactitud	Kappa	Escalamiento	Inmovilidad	Nado	Porsolt	Kappa Porsolt	Exactitud con umbral 0.80 (cobertura)
v1	8	0.691	+0.476	0.861	0.564	0.705	0.845	+0.512	0.795 (23%)
v2	8	0.707	+0.430	0.630	0.408	0.841	0.832	+0.435	0.844 (44%)
v3	8	0.775	+0.574	0.618	0.660	0.854	0.880	+0.643	0.856 (75%)
v3	16	0.781	+0.580	0.577	0.668	0.869	0.887	+0.662	0.845 (75%)
v4	16	0.804	+0.625	0.601	0.737	0.873	0.896	+0.703	0.864 (78%)

Línea base: 0.622. Matriz de confusión de v4:

Etiqueta \ CNN	Escalamiento	Inmovilidad	Nado
Escalamiento	247	14	150
Inmovilidad	13	555	185
Nado	122	121	1676

10.4 Qué se aprendió en cada paso

- **v1 → v2: no era falta de entrenamiento.** Triplicar las épocas bajó el kappa (+0.476 → +0.430). La red aprendía el aspecto de cada video (fondo, luz) y no generalizaba a los que no vio. El peso 1/frecuencia además inflaba escalamiento: 418 bloques de nado predichos como escalamiento en v1.
- **v2 → v3: quitar el fondo de la entrada fue la mejora mayor** (kappa +0.430 → +0.574). En los mapas de movimiento el fondo quieto vale casi cero. En una prueba local previa, con 3 videos y 8 épocas, el mismo cambio subió el kappa de +0.24 a +0.41 y el acierto en inmovilidad de 0.06 a 0.58.
- **8 → 16 fotogramas: casi sin efecto** (+0.574 → +0.580). La resolución temporal no era el cuello de botella.
- **v3 → v4: una CNN 3D preentrenada con video** mejoró inmovilidad (0.668 → 0.737) y la métrica Porsolt (kappa +0.662 → +0.703). El enfoque tiene precedente en las redes de dos flujos de Simonyan y Zisserman (2014), que alimentan la CNN con representaciones de movimiento.
- **Lo que queda flojo:** escalamiento (0.601), confundido con nado en 150 bloques. Es la clase con menos ejemplos (411).

11. Prueba en un video nuevo (AcuN1)

La CNN v4 se aplicó a `AcuN1`, un video que no estaba en su entrenamiento, y funcionó de punta a punta en CPU. Su acuerdo con el etiquetador en ese video fue de **0.895 (kappa +0.760)**. La exactitud contra una etiqueta independiente todavía no está medida.

Herramienta: `cnn/probar.py`. Carga `fst_cnn.pt` (pesos, tipo de entrada, número de fotogramas y versión), recorta el video con los recuadros de su `x_rasgos.csv`, predice en CPU y aplica la misma regla de propuesta de tres pasos. Escribe `x_cnn.csv` con el formato de `x_rasgos.csv`, así que `revisor.html` lo abre sin cambios. Si existe `x_mano.csv`, compara y avisa si el video estaba en el entrenamiento. La red v4 pesa 125 MB.

Procedimiento seguido:

1. `etiquetar.py` sobre `AcuN1` (4 tubos): la imagen de control mostró tubos, agua y fondo correctos.

2. `probar.py` con la CNN v4: 252 bloques.
3. Revisión en `revisor.html`, modo dudosos, sobre `AcuN1_cnn.csv`: se revisaron 11 bloques (7 confirmados, 4 corregidos) y 241 conservaron la propuesta de la CNN.

Resultados y su lectura:

Comparación	Bloques	Exactitud	Kappa	Qué mide
CNN contra <code>AcuN1_mano.csv</code>	238	0.983	+0.959	Coincidencia de la CNN consigo misma: 241 de 252 etiquetas son sus propuestas
CNN en los 11 bloques revisados	11	0.636	—	Acierto en sus bloques dudosos, que por diseño son los difíciles
Etiquetador contra <code>AcuN1_mano.csv</code>	238	0.895	+0.760	En la práctica, acuerdo entre etiquetador y CNN

El acuerdo de 0.895 entre dos métodos independientes (rasgos geométricos frente a píxeles) es la señal más informativa disponible. El mayor desacuerdo: 13 bloques que la CNN llamó nado y el etiquetador, inmovilidad.

Protocolo para una medida limpia. Etiquetar un video no visto en el modo **"desde cero"** de `revisor.html`, sin ver ninguna propuesta (unos 250 bloques, alrededor de una hora). Esas etiquetas (`origen = humano_ciego`) sirven de referencia común: se compara contra ellas la CNN y el etiquetador por igual, y ninguno de los dos habrá escrito la clave con la que se le califica.

12. Herramientas, stack y cronología

12.1 Archivos del repositorio

Archivo	Líneas	Función
<code>lib.py</code>	595	Núcleo: rotación, registro, fondo, geometría, extracción, bloques, <code>cajas_crudas</code>
<code>etiquetar.py</code>	232	Video (o <code>_rasgos.csv</code>) → rasgos, propuestas y control. Opciones <code>--modelo</code> , <code>--tubos</code> , <code>--paso</code> , <code>--umbral</code> , <code>--salida</code> , <code>--rotar</code>
<code>entrenar.py</code>	213	CSV etiquetados → <code>modelo_fst.joblib</code> , con validación por video
<code>revisor.html</code>	881	Revisión humana en el navegador; escribe <code>_mano.csv</code>
<code>comprobar.py</code>	176	Diagnóstico del entorno: versiones, archivos, carga del modelo y lectura de videos (20 sondeos por video)
<code>comprobar_videos.py</code>	134	Verifica que cada video corresponde a su CSV: existencia, duración contra el último bloque, final legible y recuadros dentro de la imagen; lista los videos aún sin etiquetar
<code>cnn/recortar.py</code>	165	Videos + etiquetas → <code>clips/*.npz</code>
<code>cnn/fst_cnn.py</code>	295	Definición, entrenamiento y validación de la CNN (versiones 1 a 4)
<code>cnn/fst_cnn.ipynb</code>	—	Cuaderno de Colab que ejecuta <code>fst_cnn.py</code> con GPU
<code>cnn/probar.py</code>	146	Aplica la CNN a un video y deja <code>_cnn.csv</code> para el revisor
<code>revisar.py</code>	—	Revisión antigua por videos cortos; reemplazada por <code>revisor.html</code>
<code>README.md</code> , <code>COMO_USARLO.md</code> , <code>CLAUDE.md</code>	—	Documentación de uso y de decisiones de diseño

No se versionan los videos (límite de 100 MB de GitHub), los clips ni los archivos `.pt`.

12.2 Stack tecnológico

Componente	Versión	Uso
Python	≥ 3.9 (probado en 3.14)	Todo
OpenCV (opencv-python)	≥ 4.8	Lectura de video, ORB, RANSAC, morfología, momentos
NumPy	≥ 2.0	Cálculo; el modelo no carga con 1.x
pandas	≥ 2.0 (probado en 3.0)	CSV y agregación
scikit-learn	1.9.x	HistGradientBoostingClassifier, GroupKFold, métricas
joblib	≥ 1.3	Guardado del modelo del etiquetador
PyTorch / torchvision	2.12 / 0.27 en local	ResNet18, R(2+1)D-18, entrenamiento de la CNN
Google Colab Pro	GPU NVIDIA T4	Entrenamiento de la CNN
Navegador Chrome o Edge	—	revisor.html
Git + GitHub	repositorio privado	Versionado y paso entre dos PC

El etiquetador no requiere GPU. La CNN requiere GPU para entrenar, pero predice en CPU.

12.3 Cronología (commits, más reciente primero)

Fecha y hora	Cambio
2026-09-26 12:00	cnn/probar.py : aplicar la CNN a un video y verla en el revisor
2026-09-26 10:21	CNN versión 4: R(2+1)D-18 preentrenada en Kinetics
2026-09-25 21:26	Número de versión visible en fst_cnn.py
2026-09-25 21:21	CNN v3: mapas de movimiento en vez de fotogramas crudos
2026-09-25 21:06	CNN v2: 40 épocas y peso de clases suavizado con raíz
2026-09-25 20:56	Cuaderno de Colab lee el código desde Drive (repositorio privado)
2026-09-25 20:43	comprobar_videos.py
2026-09-25 20:39	CNN v1: recorte de clips y entrenamiento en Colab
2026-09-25 20:17	Etiquetas revisadas de 14 videos y modelo reentrenado (0.982)
2026-09-25 17:56	--rotar para videos grabados con la cámara girada
2026-09-25 16:39	Reprocesar IMG_0826 para que el revisor pueda abrirlo
2026-09-25 16:25	Exigir numpy 2 y reentrenar solo con etiquetas humanas
2026-09-25 16:12	Conducta activa, modo desde cero y flechas de un bloque
2026-09-25 15:17	Versionar las imágenes de control
2026-09-25 14:52	revisor.html
2026-09-25 13:59	revisar.py para preparar la revisión humana
2026-09-25 11:17	Medir en segundos y no en fotogramas; candado del paso
2026-09-24 13:24	Detectar los tubos dentro de la franja de agua
2026-09-23 21:50	Preparar el repositorio para otra máquina
2026-09-23 21:40	Primer commit: etiquetador automático

13. Limitaciones y trabajo pendiente

Lo más urgente es una medida independiente: con un video etiquetado desde cero se sabrá cuánto acierta de verdad cada método.

Limitaciones conocidas:

- **Referencia sin anotador experto.** El etiquetador de referencia es una sola persona. No hay acuerdo entre observadores medido.
- **Métricas del etiquetador influidas por el flujo asistido.** La mayoría de las etiquetas de las rondas 3 a 5 son propuestas aceptadas, así que 0.982 mide sobre todo consistencia (sección 8).
- **Geometría frágil con tomas atípicas.** Falla si los rebordes del aparato no caen dentro del cuadro (IMG_0846). No hay forma de fijar la geometría a mano.
- **Cajas por bloque, no por fotograma.** Si la cámara se mueve mucho dentro de un mismo bloque de 5 s, el recorte de la CNN puede desplazarse.
- **Escalamiento sigue siendo la clase más débil de la CNN** (0.601) y la menos representada (13%).
- **Dependencias de versión:** el modelo del etiquetador exige numpy 2 y scikit-learn 1.9.

Pendiente:

- ☐ Etiquetar un video no visto en modo "desde cero" y medir contra él la CNN y el etiquetador.
- ☐ Corrida final de la CNN con `partes=14` (una por video) para reportar el acierto por video; se estima de 30 a 40 min en Colab.
- ☐ Guardar la red v4 definitiva (`fst_cnn.pt`) con la celda 5 y registrar su versión.
- ☐ Opción `--agua` y `--tubos-x` para fijar la geometría a mano en videos como IMG_0846 .
- ☐ Mejorar escalamiento en la CNN: más ejemplos de esa clase, o combinar las predicciones de v3 y v4.
- ☐ Etiquetado en vivo: la CNN es la opción viable (solo recorta y predice, con unos 5 s de retraso por bloque); requiere posiciones de tubo fijadas al inicio y cámara quieta.
- ☐ Subir al repositorio los archivos de `AcuN1` y el commit de `probar.py` , que por ahora solo están en una PC.